

How to preserve changes made inside a container

*Install Java compiler, create Java app in `/home/app`), and then ****export it for reuse and sharing**.*

1. **Commit container** → **Image** (most common for learning/labs)
2. **Export / Import vs Save / Load** (important difference)

2/11

Scenario Overview

You will:

- Start an Ubuntu container
- Install Java (javac)
- Create a Java program at `/home/app`
- Convert the modified container into an image
- Reuse & share that image

3/11

1. Run Base Ubuntu Container

```
docker run -it --name java_lab ubuntu:22.04 bash
```

Now you are **inside the container**.

4/11

2. Install Java Compiler (Inside Container)

```
apt update  
apt install -y openjdk-17-jdk
```

Verify:

```
javac --version
```

5/11

3. Create Java App in `/home/app`

```
mkdir -p /home/app  
cd /home/app
```

Create Java file:

```
nano Hello.java
```

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello from Docker container!");  
    }  
}
```

Compile and run:

```
javac Hello.java  
java Hello
```

Your container now has:

- Java installed
- Source + compiled class in `/home/app`

6/11

4. Exit Container (Changes Persist in Stopped Container)

```
exit
```

Container **stops**, but changes remain.

5. Convert Container → Image (docker commit)

This is the **key step**.

```
docker commit java_lab myrepo/java-app:1.0
```

What happened:

- Container filesystem snapshot taken
- New reusable image created

Verify:

```
docker images
```

6. Reuse the Exported Image (Locally)

Run new container from saved image

```
docker run -it myrepo/java-app:1.0 bash
```

Test:

```
cd /home/app  
java Hello
```

Java and program already exist.

7. Save / Load (Offline Transfer)

Save image to file

```
docker save -o java-app.tar myrepo/java-app:1.0
```

Transfer `java-app.tar` via USB / SCP.

Load image

```
docker load -i java-app.tar
```

Now image is available:

```
docker images
```

10/11

Important: Export vs Save (Very Common Confusion)

docker export

```
docker export java_lab > container.tar
```

- Exports **filesystem only**
- Loses:
 - Image name
 - Layers
 - CMD / ENTRYPOINT
 - Metadata

docker save (Recommended)

```
docker save -o image.tar myrepo/java-app:1.0
```

Command	Use case
<code>docker commit</code>	Container → Image
<code>docker save</code>	Image → File
<code>docker load</code>	File → Image
<code>docker push/pull</code>	Registry sharing
<code>docker export/import</code>	Raw FS only (rare)

Best Practice Note (Important we will discuss this later)

For **real projects**, prefer **Dockerfile**:

```
FROM ubuntu:22.04
RUN apt update && apt install -y openjdk-17-jdk
WORKDIR /home/app
COPY Hello.java .
RUN javac Hello.java
CMD ["java", "Hello"]
```

Build:

```
docker build -t java-app:2.0 .
```

But for **learning, labs, debugging**, `docker commit` is perfectly valid.

11/11

11. One-Line Summary

- **Modified container** → `docker commit`
- **Reuse locally** → `docker run`
- **Share online** → `docker push/pull`
- **Share offline** → `docker save/load`